

Operating Systems

Lesson Objectives

By the end of this lesson, students should be able to:

- Define what an **Operating System (OS)** is and explain its purpose.
 - Describe the **core responsibilities** of an OS.
 - Distinguish between **types of operating systems**.
 - Compare **command-line** and **graphical** user interfaces.
 - Use **Git** to manage and version-control shell script exercises.
-

1 What is an Operating System?

An **Operating System (OS)** is **system software** that manages computer hardware, software resources, and provides common services for programs.

It acts as a **bridge between the user and the computer hardware**.

Analogy:

Think of the OS as a manager that coordinates all departments (hardware, applications, users) to ensure the system runs smoothly.

Functions of an OS

1. **Resource Management** – controls how CPU, memory, and devices are used.
 2. **User Interface Management** – allows users to interact with the computer.
 3. **File and Storage Management** – organizes files and directories.
 4. **Security and Access Control** – manages user authentication and permissions.
 5. **Error Detection** – monitors and handles system errors.
-

2 Core Responsibilities of the Operating System

a) Process Management

- Handles **creation, scheduling, and termination** of processes (running programs).
- Allocates CPU time efficiently among processes.
- Ensures processes do not interfere with one another.

Example:

Running multiple browser tabs or background apps at once — the OS schedules their execution using the **CPU scheduler**.

b) Memory Management

- Tracks which parts of memory are in use and by which process.
- Allocates and deallocates memory as needed.
- Handles **virtual memory** — extending RAM using disk space.

Example:

When RAM is full, the OS uses a portion of the hard drive as “swap space.”

c) File Management

- Manages file storage, naming, and organization into directories.
- Controls **file access permissions** and keeps metadata (creation date, size, type).
- Provides commands like open, read, write, delete.

Example:

Windows Explorer or the Linux file system (/home/user/docs) rely on the file management subsystem.

d) Device Management

- Coordinates the operation of input/output devices (keyboard, printer, disk, etc.).
- Uses **device drivers** to translate OS commands into hardware signals.
- Handles interrupts (signals from devices requesting attention).

■ Example:

When you print a document, the OS sends it to the printer driver and queues it in the print spooler.

3 Types of Operating Systems

Type	Description	Examples
Batch OS	Executes batches of jobs without user interaction.	Early IBM systems
Time-Sharing OS	Allows multiple users/processes to share CPU time.	UNIX, Linux
Distributed OS	Controls multiple interconnected computers.	Google's Android cluster
Embedded OS	Built into devices with specific functions.	Smart TVs, routers
Real-Time OS (RTOS)	Processes data instantly with strict timing.	Robotics, avionics
Mobile OS	Designed for handheld devices.	Android, iOS
Network OS	Manages and supports networking functions.	Windows Server, Novell NetWare

4 User Interfaces in Operating Systems

💬 a) Command-Line Interface (CLI)

- Text-based interaction.
- Requires typing commands.
- Faster and more flexible for advanced users.

■ Examples:

- Windows Command Prompt (cmd)
- Linux Shell (bash)
- macOS Terminal

Sample Commands:

```
mkdir myfolder    # create directory
cd myfolder      # change directory
touch file.txt    # create file
rm file.txt       # delete file
```

✓ **Advantages:**

- Fast and efficient for automation.
- Useful for scripting and server administration.

⚠ **Disadvantages:**

- Requires memorizing commands.
 - Less intuitive for beginners.
-

b) Graphical User Interface (GUI)

- Uses windows, icons, menus, and pointers.
- Easier for beginners and visual navigation.

Examples:

Windows, macOS, Ubuntu Desktop, Android.

✓ **Advantages:**

- Intuitive and easy to learn.
- Suitable for general users.

⚠ **Disadvantages:**

- More resource-intensive.
 - Limited control compared to CLI.
-

Comparison: CLI vs GUI

Feature	CLI	GUI
User Interaction	Commands	Visual elements
Speed	Fast (expert users)	Slower

Ease of Use	Harder to learn	Easy to use
Automation	High (scripts)	Limited
System Resource Use	Low	High

Class Activity: Explore OS Functions

Objective: Identify OS features on your computer.

Instructions:

1. Open **Task Manager** (Windows) or **System Monitor** (Linux).
 2. Observe running processes and memory usage.
 3. Identify:
 - The process using most CPU.
 - How memory allocation changes with open apps.
 4. Document your findings in a short report.
-



Git Lab: Shell Scripts and Version Control

Task Objective:

Use Git to version control your command-line exercises.

Instructions:

1. Create a local Git repository:
2. `mkdir os_lab`
3. `cd os_lab`
4. `git init`
5. Create a simple shell script (process_check.sh):
6. `#!/bin/bash`
7. `echo "Current running processes:"`
8. `ps aux | head -5`
9. Add and commit your script:
10. `git add process_check.sh`
11. `git commit -m "Added script to list running processes"`
12. Push to GitHub:
13. `git remote add origin https://github.com/yourusername/os_lab.git`
14. `git push -u origin main`

✓ **Deliverables:**

- GitHub link to your repository.
 - Screenshot of terminal running your script.
-

